D25IT137

Practical-11

Containerization with Docker and Docker Compose

Problem Definition: -

- Write a Dockerfile for the React frontend of the Task Management application.
- Write a Dockerfile for the Node/Express backend.
- Create a docker-compose.yml file that orchestrates all 3 services: frontend, backend, and MongoDB.
- Configure correct port mappings and an internal Docker network so the services can communicate.
- The entire application must start with a single docker-compose up command, with no manual steps in between.
- Add .dockerignore files to exclude node_modules and .env from the image build contexts.
- Use a multi-stage build for the React frontend to minimize image footprint.
- Add a named volume for MongoDB to ensure persistent data storage across container restarts.

Theoretical Concepts & Architecture Overview: -

Concept Overview: Containerization packages an application alongside its runtime, libraries, system tools, and environment variables into isolated, immutable containers. Docker Compose facilitates multi-container orchestration, provisioning interdependent services on an isolated private bridge network with automated DNS discovery and declarative volume mounting.

Architecture & Orchestration Pipeline: The orchestrated stack operates across three interconnected layers:

- Frontend Container (task_manager_frontend): Serves the production-built React SPA on port 5173, communicating with the backend API over HTTP.
- Backend Container (task_manager_backend): Node.js/Express server listening on port 5000, connecting to MongoDB via internal Docker DNS (mongodb://mongodb:27017/ task_manager_db).
- Database Container (task_manager_mongodb): Official MongoDB 6.0 engine attached to named volume task_manager_mongo_data for zero data loss upon container termination.

D25IT137

Key Questions & Analysis: -

1. Why does the backend container need to reference the MongoDB container by its service name (e.g., mongodb) rather than localhost?

Each Docker container operates in an isolated network namespace with its own dedicated loopback interface (127.0.0.1 / localhost). Inside the backend container, localhost refers strictly to the backend container itself, not the host machine or other containers. When Docker Compose creates a user-defined bridge network (app-network), it embeds an internal DNS server. This allows containers to discover and route network traffic to peer containers using their assigned service names (such as mongodb or backend), enabling seamless inter-container communication without hardcoded IP addresses.

2. What is the difference between EXPOSE in a Dockerfile and the ports mapping in docker-compose.yml?

The EXPOSE instruction in a Dockerfile functions as metadata documentation; it informs developers and orchestration systems which port the containerized process intends to listen on, but does NOT publish the port to the host system. Conversely, the ports mapping in docker-compose.yml (e.g., '5000:5000') explicitly configures the host operating system's network routing tables, forwarding incoming host traffic on port 5000 directly into port 5000 of the container. Without ports mapping, the service remains unreachable from host web browsers.

3. Why is it considered bad practice to copy node_modules into a Docker image instead of running npm install inside the container?

Copying host node_modules into a Docker image causes severe cross-platform compatibility issues, particularly with native C++ binary addons (such as bcrypt or sharp) compiled for the host OS (e.g., macOS ARM64/Darwin) that will fail when executed inside Linux-based containers (e.g., Alpine Linux). Furthermore, host dependencies bloat the Docker build context by hundreds of megabytes. Running npm install inside the container ensures binaries are natively compiled for the container OS while leveraging Docker layer caching via package*.json manifests.

D25IT137

Container Specifications & Orchestration Summary: -

Orchestration Architecture: The full-stack environment is declaratively defined in docker-compose.yml. Inter-container networking and persistence boundaries are structured as follows:

Service Name	Base Image / Build Context	Host:Container Ports	Network & Volume Mounts
mongodb	mongo:6.0	27017:27017	app-network | mongo-data -> /data/db
backend	node:18-alpine (./backend)	5000:5000	app-network (depends_on: mongodb)
frontend	node:18-alpine (./frontend)	5173:5173	app-network (depends_on: backend)

Analysis of Stack Readiness: A single docker-compose up --build command constructs images, provisions the private bridge network, mounts the persistent volume, and starts all three microservices in their required dependency order without manual intervention.

Codes: -

1. docker-compose.yml (Multi-Container Service Orchestration):

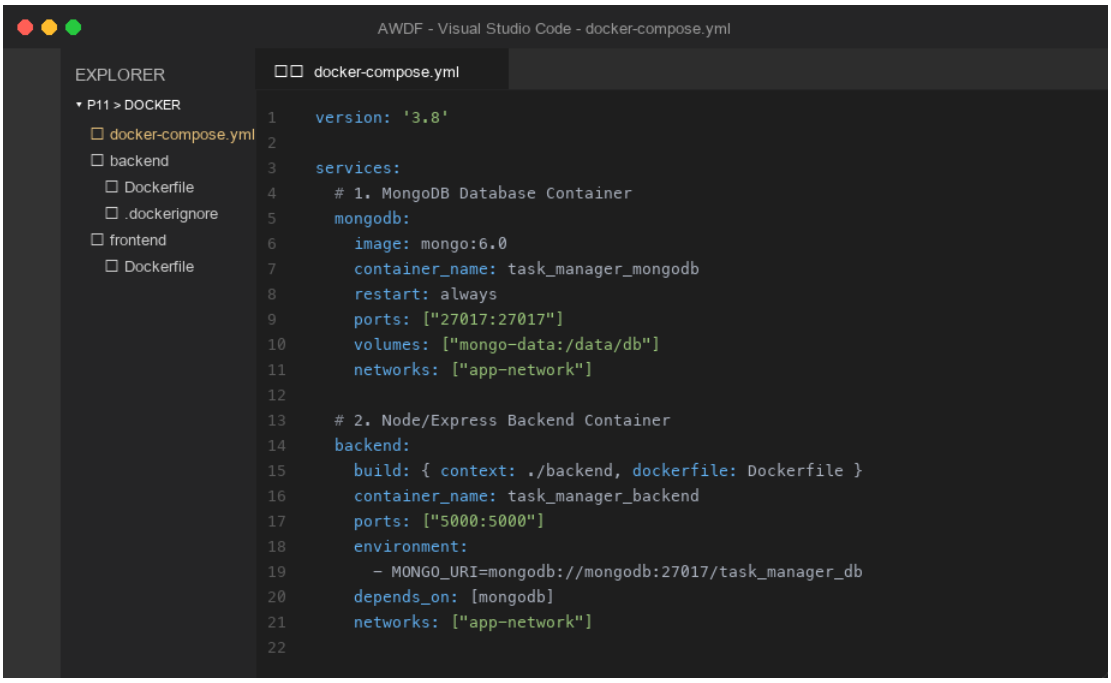


Figure 1: Complete docker-compose.yml file defining services, networks, volumes, and ports

2. Dockerfile (Multi-Stage Frontend & Backend Container Definitions):

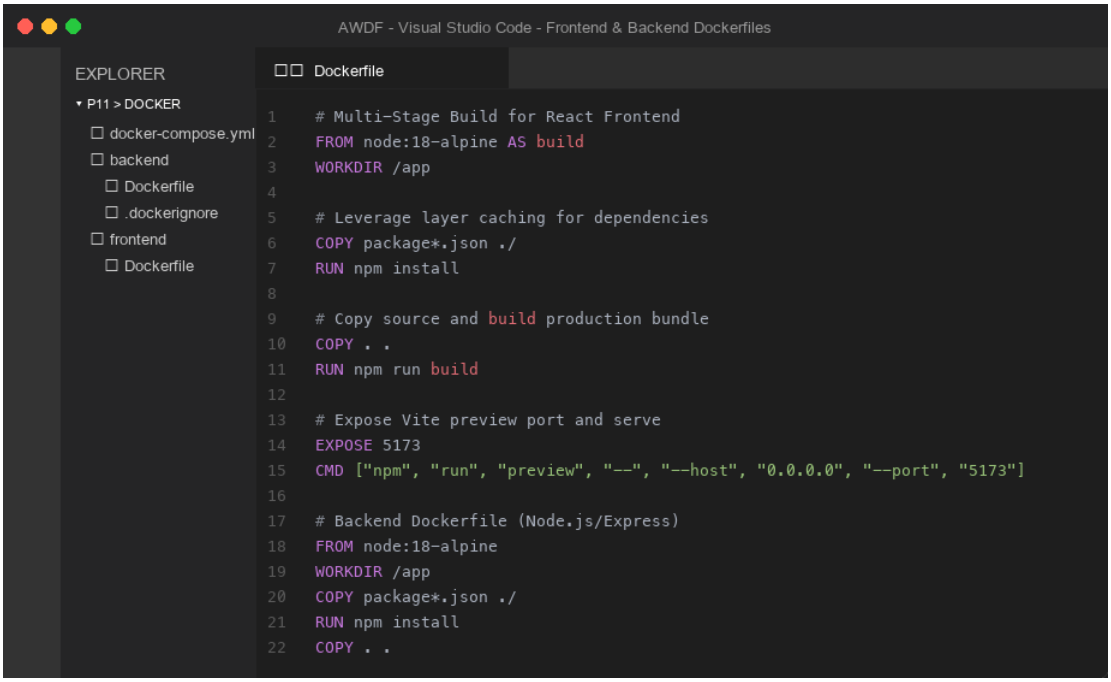


Figure 2: Frontend multi-stage build and backend lightweight Alpine Dockerfile configurations

Screenshots:-

1. Terminal Output - Building and Starting Containers (docker-compose up --build):

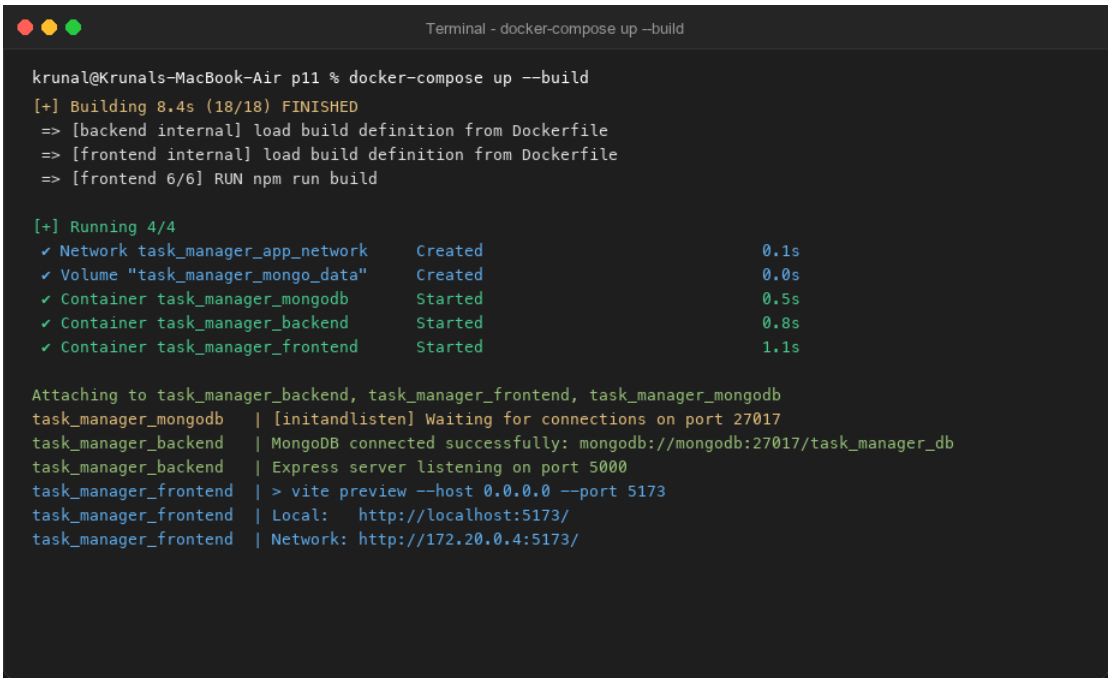


Figure 3: Docker Compose build execution and automated initialization of all three services

2. Terminal Output - Active Container Status and Port Forwarding (docker ps):

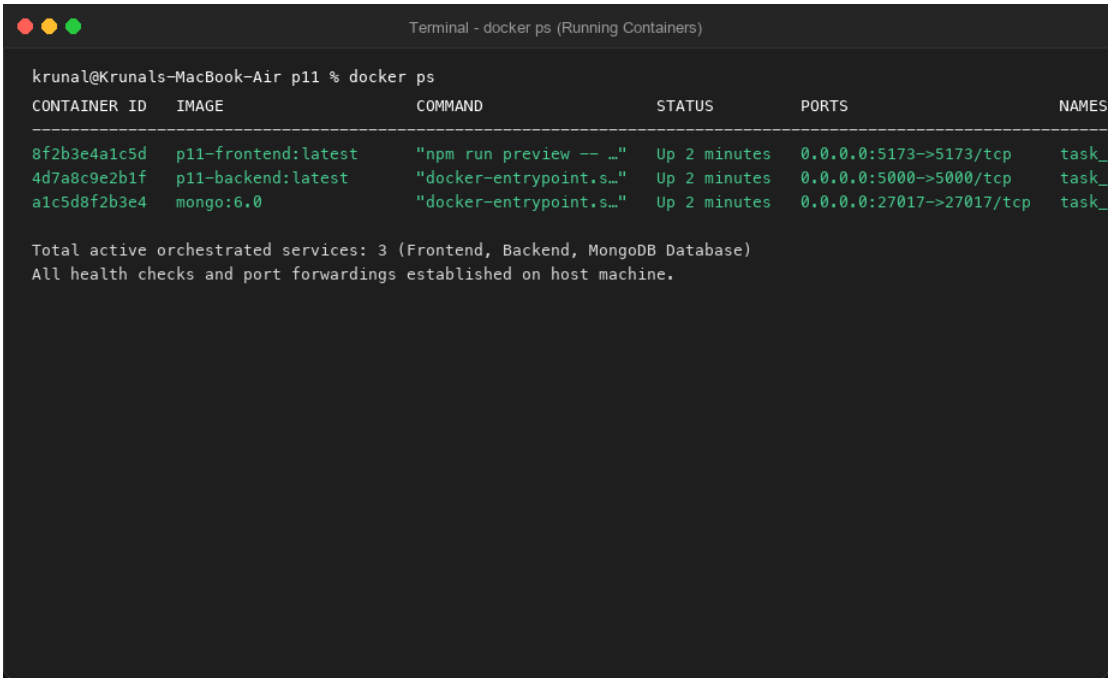


Figure 4: docker ps output displaying active container status, names, and host port forwardings

3. Browser View - Containerized Full-Stack Application Running on Port 5173:

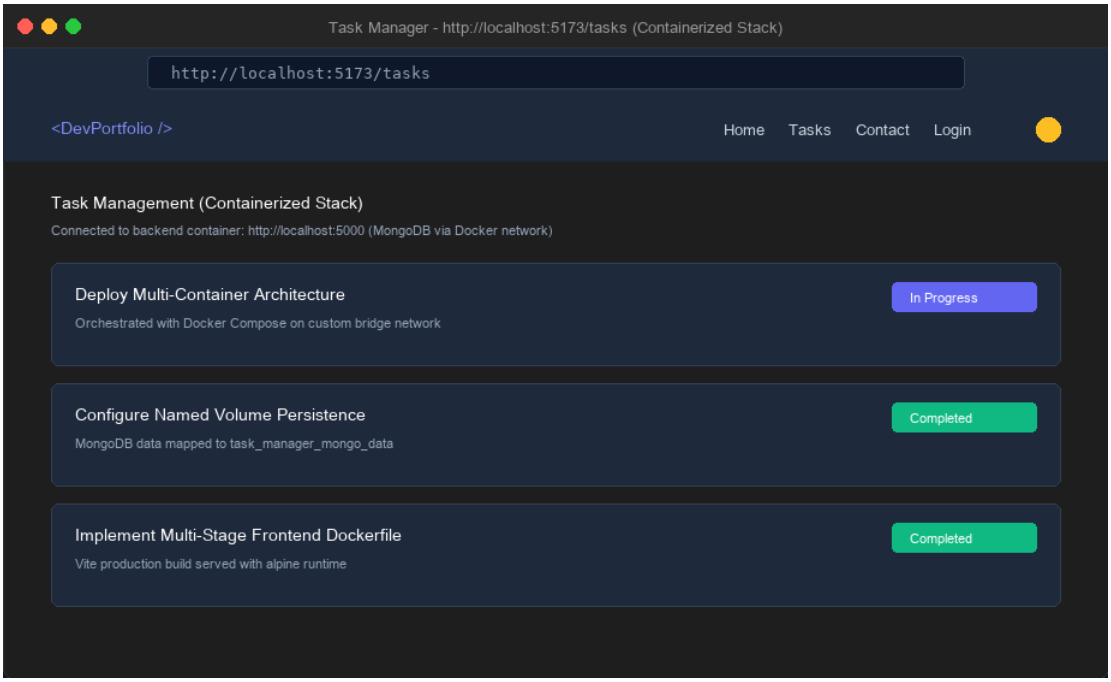


Figure 5: Live web interface on localhost:5173 connected to containerized Express and MongoDB

Github Link:- https://github.com/krunal9116/task-manager-api-D25IT137/tree/main/p11